[Accenture]

CLAUDE REGISTRY

Guida Operativa

Come usare e contribuire alle capability del team

Aprile 2026 · Team Engineering

PARTE 1

Come usare le capability

Guida per il developer che vuole integrare e usare le capability nel proprio progetto.

Prerequisiti

Prima di iniziare, assicurati di avere i seguenti strumenti disponibili nel tuo ambiente di sviluppo:

- Claude Code installato (`claude.ai/code` oppure tramite IDE extension)
- Accesso al repository: `github.com/luketherose/claude-registry`
- Python 3 disponibile — verifica con `python3 --version`

Installazione rapida (script)

Il modo più veloce per integrare le capability nel tuo progetto è usare lo script di setup:

```
git clone https://github.com/luketherose/claude-registry.git ~/dev/claude-registry
cd ~/dev/claude-registry
./claude-catalog/scripts/setup-capabilities.sh /path/to/tuo-progetto
```

Lo script mostra un menu interattivo: seleziona i numeri delle capability desiderate oppure digita `all` per installare tutte le stable, poi `q` per uscire.

> **Risultato:** i file `.md` delle capability selezionate vengono copiati nella directory `.claude/agents/` del progetto, e il file `settings.json` viene aggiornato automaticamente con i permessi necessari.

Verifica dell'installazione

Dopo l'installazione, puoi verificare che le capability siano disponibili in due modi:

Via comando Claude Code

```
# In Claude Code:
/agents   # mostra le capability disponibili
```

Via task naturale

Avvia un task in linguaggio naturale e Claude delega automaticamente al subagent appropriato:

```
"Puoi fare una review dell'architettura?"
→ Claude delega automaticamente a software-architect
```

Capability disponibili

Di seguito l'elenco completo delle capability presenti nel registry con il tier di maturità e il caso d'uso principale.

Capability	Tier	Quando usarla
`software-architect`	stable	Analisi architetturale, ADR, trade-off tra alternative
`functional-analyst`	stable	Requisiti, use case, processi di business
`developer-java-spring`	stable	Sviluppo Java/Spring Boot enterprise
`technical-analyst`	beta	Debito tecnico, sicurezza, dipendenze vulnerabili
`developer-python`	beta	Sviluppo Python/FastAPI
`code-reviewer`	beta	Code review strutturata su PR o file
`test-writer`	beta	Test JUnit 5, Mockito, Testcontainers, pytest
`debugger`	beta	Diagnosi bug da stack trace, log, codice
`api-designer`	beta	Design e review API REST, spec OpenAPI
`documentation-writer`	beta	README, runbook, guide architetturali
`presentation-creator`	beta	Presentazioni PPTX Accenture-branded da documenti
`document-creator`	beta	Documenti PDF/DOCX Accenture-branded da documenti

Installazione manuale (senza script)

Se preferisci un approccio più diretto o hai bisogno di installare una singola capability, puoi farlo copiando manualmente il file:

```
mkdir -p .claude/agents
cp ~/dev/claude-registry/claude-marketplace/stable/software-architect.md
.claude/agents/
```

Ripeti il comando per ogni capability che vuoi aggiungere, puntando alla directory `stable/` o `beta/` a seconda del tier.

Configurazione settings.json (opzionale)

Per gestire esplicitamente i permessi di delega, puoi configurare il file `.claude/settings.json` del progetto:

```
{
  "permissions": {
    "allow": [
      "Agent(software-architect)",
      "Agent(functional-analyst)"
    ]
  }
}
```

> Questo passaggio è opzionale: lo script di setup aggiorna `settings.json` automaticamente per le capability selezionate.

Personalizzazione per il progetto

Ogni capability può essere personalizzata per il contesto specifico del progetto:

1. Copia il file `.md` nella directory `.claude/agents/` del progetto.
2. Rinomina il file con il nome desiderato (es. `software-architect-myproject.md`).
3. Aggiorna il campo `name` nel frontmatter per corrispondere al nuovo filename.
4. Aggiungi il contesto specifico del progetto in fondo al file — tecnologie adottate, vincoli architetturali, convenzioni del team.

La personalizzazione permette al subagent di rispondere in modo più preciso al contesto del progetto senza modificare la capability originale nel registry.

Aggiornamento delle capability

Per aggiornare le capability già installate all'ultima versione disponibile nel registry:

```
cd ~/dev/claude-registry && git pull
./claude-catalog/scripts/setup-capabilities.sh /path/to/tuo-progetto
# riseleziona le capability da aggiornare
```

Lo script sovrascrive i file esistenti in `.claude/agents/`. Se hai personalizzato una capability, esegui un diff prima di aggiornare per non perdere le modifiche locali.

Uso del CLI `present` (presentazioni e documenti)

Lo script `present.sh` genera presentazioni PPTX o documenti PDF branded Accenture a partire da una directory di documenti sorgente (stime, requisiti, architettura, ecc.).

Parametri principali

Parametro	Valori	Descrizione
`--type`	`pptx`, `pdf`	Formato di output
`--audience`	`biz`, `tech`	Target del documento — influenza tono e livello di dettaglio
`--project`	stringa	Nome del progetto (opzionale, appare nel titolo)
`--output`	path	Percorso di output (opzionale, default: directory corrente)

Esempi d'uso

Presentazione business

```
./claude-catalog/scripts/present.sh --type pptx --audience biz ./estimation/
```

Documento tecnico PDF

```
./claude-catalog/scripts/present.sh --type pdf --audience tech ./estimation/
```

Con opzioni complete

```
./claude-catalog/scripts/present.sh \
  --type pptx \
  --audience tech \
  --project "Portal Modernization" \
  --output ~/Desktop/deck.pptx \
  ./estimation/
```

PARTE 2

Come contribuire

Guida per il membro del team che vuole aggiungere o migliorare una capability nel registry.

Ciclo di vita di una capability

Ogni capability segue un ciclo di vita controllato prima di essere disponibile ai team:

1. **Draft** — il contributor crea la capability localmente con lo script `new-capability.sh`
2. **Review PR** — apertura della pull request su GitHub
3. **GitHub Actions CI** — check automatici sul frontmatter e sulla struttura del file
4. **Review manuale** — revisione da parte di un maintainer del team
5. **Merge** — approvazione e merge su `main`
6. **Pubblicazione marketplace** — script `publish.sh` pubblica nel marketplace
7. **In uso dai team** — disponibile per l'installazione nei progetti

Creare una nuova capability (script)

Per iniziare a sviluppare una nuova capability, usa lo script dedicato:

```
cd ~/dev/claude-registry
./claude-catalog/scripts/new-capability.sh nome-capability
```

Lo script crea automaticamente tutti i file necessari e prepara l'ambiente di sviluppo:

- `claude-catalog/agents/nome-capability.md` — system prompt con template e TODO
- `claude-catalog/examples/nome-capability-example.md` — esempi di interazione
- `claude-catalog/evals/nome-capability-eval.md` — scenari di valutazione
- Branch git `add/nome-capability` creato automaticamente
- Entry in `CHANGELOG.md` aggiunta automaticamente

Scrivere il system prompt

Il file principale è `claude-catalog/agents/nome-capability.md`. La struttura deve rispettare un formato preciso.

Frontmatter obbligatorio

```
---
name: nome-capability
description: >
  Use when ... (condizione di trigger precisa)
tools: Read, Grep, Glob
model: sonnet
color: blue
---
```

> Il campo `description` deve descrivere la condizione di trigger in modo preciso, non vago. GitHub Actions verifica che `name` corrisponda al filename del file.

Sezioni obbligatorie nel body

Sezione	Contenuto atteso
`## Role`	Chi è il subagent, su cosa è autorevole, quali decisioni prende
`## What you always do`	Comportamenti obbligatori — regole che il subagent deve sempre rispettare
`## What you never do`	Proibizioni esplicite — cosa il subagent non deve mai fare
`## Output format`	Formato di output esatto con esempi concreti
`## Quality self-check`	Controlli che il subagent deve eseguire prima di rispondere

Testare localmente

Prima di aprire la PR, verifica il comportamento della capability in un progetto di test:

```
cp claude-catalog/agents/nome-capability.md /path/to/test-
project/.claude/agents/
# Apri Claude Code nel test project
# Esegui gli scenari definiti nel file di eval
```

Per ogni scenario negli eval, verifica che:

- Il subagent viene invocato correttamente dal routing automatico di Claude
- L'output rispetta il formato definito nella sezione `## Output format`
- I comportamenti proibiti nella sezione `## What you never do` non si verificano
- Il quality self-check produce output coerente

Aprire la Pull Request

Quando la capability supera i test locali, apri la PR:

```
git add -A
git commit -m "add: nome-capability capability (draft)"
git push origin add/nome-capability
gh pr create --title "add: nome-capability" \
  --body "Aggiunge la capability nome-capability per [descrizione uso]"
```

> Usa il prefisso `add:` per nuove capability e `update:` per modifiche a capability esistenti. Il titolo della PR viene usato nel CHANGELOG.

Review automatica GitHub Actions

Dopo il push, GitHub Actions esegue automaticamente i check sul file della capability e posta un commento sulla PR con i risultati.

Errori che bloccano il merge

- Il campo `name` nel frontmatter non corrisponde al filename del file
- Campo `description` assente o vuoto
- Tools dichiarati non validi o non riconosciuti
- Sezioni obbligatorie mancanti nel body

> La PR può essere approvata solo dopo che tutti gli errori segnalati da GitHub Actions sono stati risolti.

Checklist review manuale

Il maintainer verifica i seguenti punti prima di approvare la PR (definiti in `review-checklist.md`):

Punto	Criterio
`name`	Corrisponde esattamente al filename (senza estensione)
`description`	Precisa e specifica — non generica o vaga
`tools`	Minimali e necessari — nessuno strumento dichiarato per eccesso
Role	Chiaramente definito, senza sovrapposizioni con altre capability
Output format	Contiene esempi concreti, non solo descrizioni astratte
Examples	Almeno 1 esempio di interazione completo
Eval scenarios	Almeno 2 scenari di valutazione con input e output atteso
CHANGELOG.md	Aggiornato con la nuova entry nella sezione corretta

Pubblicare nel marketplace

Dopo il merge su `main`, la capability viene pubblicata nel marketplace tramite lo script `publish.sh`:

Prima pubblicazione (beta)

```
./claude-marketplace/scripts/publish.sh nome-capability 0.1.0 beta
```

Promozione beta → stable

Una capability può essere promossa a `stable` dopo almeno 30 giorni di uso e adozione in almeno 2 progetti:

```
./claude-marketplace/scripts/publish.sh nome-capability 1.0.0 stable
```

Versioning (SemVer)

Le capability seguono il versionamento semantico. La scelta della versione dipende dalla natura della modifica:

Tipo	Esempio	Quando usarlo
PATCH	`1.0.0 → 1.0.1`	Fix al prompt, nessun cambio di comportamento osservabile
MINOR	`1.0.0 → 1.1.0`	Nuovo comportamento aggiunto, retrocompatibile con l'esistente
MAJOR	`1.0.0 → 2.0.0`	Cambio di `name`, `description`, o `tools` — breaking change

> Un cambio MAJOR richiede comunicazione proattiva ai team che usano la capability, poiché potrebbe richiedere aggiornamenti nei loro progetti.

Migliorare una capability esistente

Per modificare una capability già nel registry, il flusso è simile ma usa il prefisso `update/` per il branch:

```
git checkout -b update/nome-capability
# Edita claude-catalog/agents/nome-capability.md
# Aggiorna CHANGELOG.md con la descrizione della modifica
git push origin update/nome-capability
gh pr create --title "update: nome-capability — [descrizione modifica]"
```

Anche le modifiche a capability esistenti passano per GitHub Actions CI e review manuale prima del merge.

© 2026 Accenture. All rights reserved.